

Taste Matcher

Agile Sprint Tracker

Developer & QA Lead: Srirama Prabhav R.

Repository: github.com/thatmanspr/letterboxd-taste-matcher

Sprint History Overview

Sprint	Release	Date	Focus	Tests	Status
Sprint 1	v1.0	—	Core prototype — scoring pipeline, CSV ingestion, basic UI	—	✓ Shipped
Sprint 2	v2.1	May 4, 2026	Data integrity refactor, unified scoring, export compliance	86	✓ Shipped
Sprint 3	v3.0	May 4, 2026	Electron desktop app, full UI overhaul, live mutations	—	✓ Shipped
Sprint 4	v3.1	May 5, 2026	Ranking priority system, bug fixes	—	✓ Shipped

Sprint 1 — v1.0 Prototype

Sprint Goal: Build a working end-to-end watchlist ranker from Letterboxd CSV exports using TMDb metadata and a multi-dimensional taste model.

Scope Delivered

- Node.js + Express backend reading ratings.csv and watchlist.csv from /data/
- TMDb API integration — genres, directors, writers, countries, decades, keywords, collections
- Multi-dimensional taste profile builder (genre, director, writer, country, region, decade, keyword, collection)
- Nearest-neighbour similarity layer using Jaccard similarity over genres, keywords, decade, regions, directors
- Final weighted score formula (predictedScore across 9 components, weights sum to 1.00)
- Disk cache (tmdb_cache.json, derived_cache.json) — TMDb data fetched once, never again
- Basic frontend: ranked watchlist cards, rewatch list, genre profile with click-to-expand
- .env-based config (TMDb key + PORT)
- Export ranked watchlist as Letterboxd-compatible list CSV

Known Issues Carried into Sprint 2

- Overlap detection relied solely on title+year, causing missed rewatches when URL formats differed
- scoreItem() logic was duplicated — the export endpoint used a separate code path, producing a different order than the UI

- Rewatch ranking formula ($0.6 \times \text{rating} + 0.4 \times \text{model}$) allowed low-rated films to outrank high-rated ones if the model disagreed strongly
- No duplicate prevention — add-rating and add-to-watchlist could create phantom duplicates when films were added via different identifiers
- Export used an incorrect Letterboxd CSV format and wrote to disk rather than streaming a browser download
- `derived_cache.json` did not invalidate stale entries missing `ratedItemsMeta`, causing silent neighbour component failures on restart

Sprint 2 — v2.1 Refactor

Sprint Goal: Resolve v1 data integrity flaws (duplicate entries), implement Letterboxd v7 export compliance, and consolidate the scoring architecture.

Date: May 4, 2026

Sprint Retrospective

Sprint 2 addressed the technical debt accumulated during the v1 prototype phase, specifically around state management, API bottlenecking, and data ingestion overlap. TDD principles guided the refactor, resulting in a unified `scoreItem()` function and robust caching mechanisms. 86 functional test cases were manually executed against the v2 release candidate, focusing on mutation endpoints and edge-case handling.

Agile Epics & Test Coverage

- Epic 1 — Data Pipeline & Ingestion (Tests ST-01 to CSV-06): Universal CSV parsing, header detection fallback, zero-downtime hot-reloading of watchlist.
- Epic 2 — The Taste Model Engine (Tests TM-01 to RW-06): k-NN similarity layer, Bayesian-smoothed profiles, revised weighted heuristics. Verified Keyword Affinity weight at 0.22.
- Epic 3 — UI/UX & State Filtering (Tests WL-01 to WL-10, UI-01 to UI-10): Real-time DOM updates, lazy loading for TMDb poster assets, multi-factor list filtering (title, year, decade, director).
- Epic 4 — State Mutations & Export (Tests MU-01 to EX-07): Triple-validation duplicate prevention (URL, Title+Year, TMDb ID). Letterboxd v7 CSV compliance and native browser download.

Test Execution Summary

Testing Area	Test Cases	Status	Notes
System Boot & Cache Persistence	12	✓ PASS	JSON flat-file state recovery verified
Data Parsing & Overlap Detection	6	✓ PASS	Title+Year fallback catches edge-case duplicates
Scoring Algorithm & Rewatch Math	14	✓ PASS	Bayesian smoothing and anchor metrics functioning
UI Rendering & Filters	20	✓ PASS	Front-end handles ranked lists without framework overhead

Testing Area	Test Cases	Status	Notes
Mutations & Export Compliance	20	✓ PASS	v7 Letterboxd format confirmed; browser download native
Edge Cases & Error Handling	14	✓ PASS	Graceful degradation confirmed for missing assets
TOTAL	86	✓ PASS	Ready for Deployment

Bugs Fixed

Fix	Description
Overlap detection	URL match is now the primary check; title+year is the fallback. Films with different URL formats but matching title+year are correctly identified as rewatches.
Unified scoring function	A single shared scoreItem() is used by watchlist ranking, rewatch ranking, and export. Previously, the export used a separate code path and could return a different order than the UI.
Rewatch ranking formula	Old formula ($0.6 \times \text{rating} + 0.4 \times \text{model}$) allowed low-rated films to rank above high-rated ones. New formula: $\text{rewatchScore} = \text{userRatingNorm} \times (0.70 + 0.30 \times \text{modelScore})$
Triple duplicate check	add-rating and add-to-watchlist now check for duplicates by URL, title+year, and TMDb ID, preventing phantom duplicates.
Export format	Export now uses the correct Letterboxd list v7 format and streams as a direct browser download instead of writing to disk.
Derived cache invalidation	Old derived_cache.json entries missing ratedItemsMeta are now detected on load and discarded, forcing a clean rebuild.

Backlog Items Promoted to Sprint 3

1. API Concurrency (Performance): TMDb fetching runs synchronously. Sprint 3 to implement Promise.all() batching for large CSV imports.
2. Customizable Recommendation Engine (UI/UX): Allow users to override default dimension weights via the front end.
3. Machine Learning Integration (Scoring): Replace static linear heuristics with Matrix Factorization or a lightweight neural network trained on the full ratings dataset.

Sprint 3 — v3.0 Desktop App

Sprint Goal: Graduate Taste Matcher from a dev-server tool to a polished cross-platform desktop application. Introduce an Electron wrapper, a full UI overhaul, and a suite of live-editing features.

Scope Delivered

Electron App

- Full Electron wrapper — runs as a native desktop app on Windows, macOS, and Linux
- Server runs in-process (no separate terminal required)

- First-run setup wizard (3-step: ratings CSV → watchlist CSV → TMDb key)
- Per-platform user data directory (%APPDATA%, ~/Library/Application Support, ~/.config)
- config.json persists TMDb key and CSV paths; all data lives outside the app bundle
- IPC bridge (preload.js) exposes file-picker and CSV-install to the renderer securely
- npm run dist / npm run dist:win build distributable installers

New UI Features

- Redesigned full-page frontend — dark theme, card grid, action bar, settings drawer
- Settings drawer: live app stats, TMDb connection badge, CSV reload buttons, reset and back-to-setup actions
- Toggle between card view and list view
- Change rating — star picker opens pre-filled with existing rating; saving immediately invalidates taste model and ranked watchlist caches
- Sort rated films by highest or lowest rating (alphabetical tiebreak)
- ⌚ Recalculate button on the Ranked Watchlist — forces a fresh re-score without any new TMDb calls
- Ratings added manually now automatically trigger a watchlist recalculation

New Backend Features

- update-rating endpoint — change an existing rating with full cache invalidation
- invalidate-recommendations endpoint — decouple cache-busting from UI reloads
- search-ratings endpoint with ?all=true support
- app-status endpoint for the settings drawer stats panel
- validate-tmdb-key endpoint for the setup wizard test button
- data_state.json persistence — in-memory mutations survive restarts
- hidden_rewatches.json persists rewatch hides across restarts
- Failed items (unresolved TMDb matches) tracked and surfaced in the UI with manual resolution flow
- Export modal — choose watchlist only or include rewatches before downloading

QA Notes

Full manual regression pass against all Sprint 2 test cases confirmed no regressions. New functionality covered via exploratory testing across all mutation flows and Electron IPC paths on Windows.

Backlog Items Carried Forward

- API concurrency (Promise.all() batching) — descoped in favour of UX work; remains in backlog
- Customizable recommendation weights — partially satisfied; full implementation promoted to Sprint 4
- ML integration — remains in long-term backlog

Sprint 4 — v3.1 Ranking Priorities

Sprint Goal: Deliver the customizable recommendation engine backlog item from Sprint 2. Let users control how the nine scoring dimensions are weighted against each other, surfaced through a first-run priority screen and a persistent settings entry.

Date: May 5, 2026

Scope Delivered

Ranking Priority System (Frontend)

- First-launch intercept: priority modal opens before scoring begins when weights are still at defaults
- Modal presents all 9 scoring dimensions as draggable cards, each with a plain-English description (no weights or percentages shown)
- Drag-and-drop reordering with live rank numbers updating as cards move
- ▲/▼ arrow button fallback for keyboard/accessibility use
- ⌛ Use Defaults resets to the original algorithm distribution instantly
- Save & Rank Watchlist → saves the priority order and immediately kicks off `loadRecommendations()`
- Priority modal accessible at any time via ⚙ Settings → Ranking Priorities
- After saving from settings, watchlist re-renders immediately (no ⌛ Recalculate needed)

Ranking Priority System (Backend)

- `user_weights.json` — new cache file storing the user's active weight distribution, persists across sessions
- `DEFAULT_WEIGHTS` constant — original hardcoded values extracted (genre 0.18, keyword 0.22, director 0.10, writer 0.08, tmdb 0.15, neighbour 0.10, geo 0.07, decade 0.05, collection 0.05)
- `weightsFromPriorityRanking(ranking)` — converts a priority-ordered array into a full weight distribution using geometric decay (each rank = $0.75 \times$ the previous); total always sums to 1.00
- `activeWeights` — runtime variable loaded from disk on startup, overrides `DEFAULT_WEIGHTS` when a user preference exists
- `scoreItem()` updated to read from `activeWeights` instead of hardcoded literals
- Three new API endpoints: GET `/api/user-weights`, POST `/api/user-weights`, POST `/api/user-weights/reset`
- `loadUserWeightsFromDisk()` called at server startup alongside existing cache loaders

Bug Fixes

- Fixed: saving priorities from the Settings drawer (non-first-launch path) did not re-render the ranked watchlist — the conditional else branch called `invalidate` but never called `loadRecommendations()`. Removed the conditional; save and reset now always reload immediately.
- Fixed: server-side error responses from POST `/api/user-weights` were silently swallowed by the frontend; save handler now checks `result.error` and surfaces it as a toast.

Test Coverage

Area	Tested	Notes
First-launch modal intercept	✓	Only shows when weights are at defaults; skips if user has saved preferences
Drag-and-drop reorder	✓	All 9 dimensions, all positions
Arrow button reorder	✓	Up/down at boundaries (disabled correctly)
Save → immediate reload	✓	Rankings change on save, no manual recalculate needed
Reset → immediate reload	✓	Weights revert to defaults, rankings update
Settings path (non-first-launch)	✓	Modal opens, save triggers reload correctly after bug fix
user_weights.json persistence	✓	Priority order survives app restart
scoreItem() weight injection	✓	Confirmed different priority orders produce different ranking outputs
weightsFromPriorityRanking math	✓	Sum = 1.00 across all test orderings; geometric decay verified
Server error surfacing	✓	Invalid ranking array returns 400; frontend shows error toast

Backlog Items for Sprint 5

4. API Concurrency: Promise.all() batching for TMDb fetches during first-run CSV import. Still the highest-value remaining performance item.
5. ML Integration: Matrix Factorization or lightweight neural net to replace static linear scoring. Long-term; requires dataset tooling first.
6. Per-dimension weight fine-tuning: Advanced mode allowing numeric slider control within a priority tier, for power users who want more granularity than rank order alone.